Compte rendu : cédule de maintenance

Farah Akrout · LSI 3 · Algorithme génétique

1. Objectif

L'exercice consiste à planifier la maintenance de sept unités de production sur quatre intervalles de l'année, en gardant la réserve de puissance la plus grande possible. Quand le nombre de cédules possibles augmente, les méthodes exactes deviennent coûteuses. On utilise donc un algorithme génétique, qui fait évoluer une population de cédules par sélection, croisement et mutation.

2. Données du problème

Unité	Capacité (MW)	Intervalles de maintenance
1	20	2
2	15	2
3	35	1
4	40	1
5	15	1
6	15	1
7	10	1

La capacité totale est de 150 MW. L'année est découpée en 4 intervalles, avec des demandes de 80, 90, 65 et 70 MW. La maintenance d'une unité commence au début d'un intervalle et se termine à la fin d'un intervalle. La réserve de chaque intervalle ne doit pas être négative.

3. Modélisation

- **Chromosome :** une cédule complète, formée de 7 gènes de 4 bits (un gène par unité). Un bit à 1 signifie que l'unité est en maintenance pendant l'intervalle correspondant.
- **Gènes valides :** pour les unités 1 et 2 : 1100, 0110, 0011. Pour les unités 3 à 7 : 1000, 0100, 0010, 0001. Ce choix garantit que les intervalles de maintenance sont consécutifs.
- **Réserve d'un intervalle :** capacité totale − puissance en maintenance − demande.

- **Fitness :** la plus petite des quatre réserves, ramenée à 0 si elle est négative. On cherche à la maximiser.

Exemple du cours. Pour le chromosome 0110 0011 0001 1000 0100 0010 1000, la puissance en maintenance vaut 50, 35, 50 et 50 MW. Les réserves sont donc 150 − 50 − 80 = 20, 150 − 35 − 90 = 25, 150 − 50 − 65 = 35 et 150 − 50 − 70 = 30. La fitness est la plus petite, soit 20 MW.

4. Algorithme génétique utilisé

1. **Population initiale :** 50 chromosomes valides tirés au hasard.
2. **Évaluation :** calcul des réserves puis de la fitness de chaque individu.
3. **Sélection par loterie (roulette) :** la probabilité de choisir un individu est proportionnelle à sa fitness. Si toutes les fitness sont nulles, le tirage est uniforme pour éviter une division par zéro.
4. **Croisement :** à un point de coupure, toujours placé entre deux gènes de 4 bits. Couper à l'intérieur d'un gène donnerait des cédules illogiques.
5. **Mutation :** toujours appliquée à un gène entier, pour garder la cédule valide. Trois variantes sont comparées :
   - *inversion* : l'ordre des 4 bits est inversé (1100 devient 0011). Limite : 0110 ne change jamais ;
   - *remplacement* : le gène est remplacé par une autre configuration valide de la même unité (0001 devient 0100, comme sur la diapo) ;
   - *échange* : les gènes de deux unités ayant la même durée de maintenance sont permutés.
6. **Élitisme :** le meilleur individu est toujours conservé dans la génération suivante.
7. **Arrêt :** après 100 générations. Le meilleur individu rencontré est affiché, la dernière génération comprise.

5. Paramètres

Paramètre	Valeur
Taille de la population	50
Nombre de générations	100
Probabilité de croisement	0,8
Probabilité de mutation	0,1

La taille de population respecte la règle du cours : 6 à 7 fois le nombre de variables, soit 42 à 49 pour 7 unités, arrondi à 50. Comme le cours conseille de tester plusieurs valeurs, Pc (0,6 ; 0,8 ; 0,9) et Pm (0,05 ; 0,1 ; 0,2) ont aussi été comparées.

6. Correspondance avec le code (cedule.py)

Notion	Fonction
Population	creer_population()
Réserves	calculer_reserves()
Fitness	fitness()
Sélection	selection_loterie()
Croisement	croisement()
Mutations	mutation_inversion(), mutation_remplacement(), mutation_echange()
Boucle des générations	algorithme_genetique()
Vérification de l'optimum	optimum_force_brute()

7. Résultats

Exemple du cours : fitness = 20 MW.

Optimum exact (force brute). L'espace de recherche contient $3 \times 3 \times 4^5 = 9\,216$ cédules, ce qui permet de toutes les tester. La meilleure fitness est de 25 MW. Par exemple, la cédule 0011 0011 0100 1000 0010 0001 0010 donne des réserves de 30, 25, 25 et 30 MW.

Cette valeur est bien un maximum : la somme des quatre réserves vaut 4 × 150 − 185 (puissance totale en maintenance) − 305 (demandes totales) = 110 MW, donc la plus petite réserve ne peut pas dépasser 27,5 MW. Comme toutes les valeurs sont des multiples de 5, le maximum est 25.

Comparaison des mutations (30 exécutions chacune) :

Mutation	Moyenne	Minimum	Optimum trouvé
Inversion	[à compléter]	[à compléter]	[à compléter] / 30
Remplacement	[à compléter]	[à compléter]	[à compléter] / 30

Mutation	Moyenne	Minimum	Optimum trouvé
Échange	[à compléter]	[à compléter]	[à compléter] / 30

Influence de Pc et Pm (mutation par remplacement, 10 essais) :

Pc \ Pm	0,05	0,1	0,2
0,6	[à compléter]	[à compléter]	[à compléter]
0,8	[à compléter]	[à compléter]	[à compléter]
0,9	[à compléter]	[à compléter]	[à compléter]

8. Analyse

- L'algorithme génétique atteint rapidement des cédules de bonne qualité, et l'encodage par blocs de 4 bits garantit que le croisement et la mutation ne produisent jamais de cédule invalide.
- La mutation par inversion explore peu : pour une unité à un seul intervalle, elle ne fait qu'échanger 1000 avec 0001 et 0100 avec 0010, et elle ne modifie jamais 0110. [à compléter : ce que montre ton tableau de comparaison]
- [à compléter : les valeurs de Pc et Pm qui donnent les meilleurs résultats d'après ton tableau]
- Ici, la recherche exhaustive reste possible car il n'y a que 9 216 cédules. Elle deviendrait impraticable avec plus d'unités ou d'intervalles, alors que l'algorithme génétique s'adapte à des espaces beaucoup plus grands.

9. Conclusion

Ce travail applique les notions du cours (population, individu, gène, fitness, sélection, croisement, mutation, générations) à un problème réel d'ordonnancement. L'algorithme génétique trouve une cédule qui maximise la plus petite réserve de puissance tout en respectant les contraintes, et la force brute confirme que l'optimum est de 25 MW.